

Reply: fork-choice tiebreak implemented;

4b left out

Done, exactly as you decided.

1. Lower-hash tiebreak after the VRF check: implemented

Among same-height candidates the fork-choice comparator now orders by (1) most countersignature power, (2) lowest leader VRF score, and (3) as the final key, lowest block hash. The hash key sits AFTER the VRF check and replaces the old node-local first-seen (`nSequenceId`) step for PoS blocks. Because the block hash is globally deterministic, every honest node breaks a residual tie identically, so two equally-certified same-height blocks can no longer persist as a split. That first-seen fallback was the only non-deterministic step left in the fork choice; it is now closed.

The key sits strictly below total work and countersignature power, so it never reorders across work, certification, immediate finality, or a Bitcoin-driven reorg. It only decides among blocks that already tie on all of those. No anchor key in the fork choice, as before.

2. 4b reconciliation: left out, and I agree

No reconciliation or convergence rule was added. Your reasoning is right and worth stating plainly: the asymmetric stall costs liveness, not safety; it self-resolves in the common case and is trivially operator-recoverable otherwise. A height- or frequency-based reconciliation would trade that rare liveness cost for a real safety hole, letting an attacker outrun the honest chain from the last checkpoint just by spamming certified blocks at a higher frequency to reach a greater height. Not worth it. The deterministic hash tiebreak gives every node one answer without opening that door.

3. One behavior change worth flagging

`preciousblock` no longer overrides a PoS fork-choice tie. It works by lowering a block's `nSequenceId`, which now sits below the deterministic hash key, so for equal-work, equal-certification, equal-VRF-score PoS blocks the hash decides regardless. This is consistent with the design (deterministic convergence replaces manual preference), and operator recovery of a stuck tip is unaffected: `invalidateblock` on the losing tip and `reconsiderblock` act on validity, not on sequence order.

Status

Committed and pushed (`fc06176a6`); `elements` builds clean. I ran a six-lens adversarial review (ordering soundness, network convergence, anchoring and finality interaction, attack surface including hash-grinding and the 51 percent certified-block spam vector you flagged, directive compliance, and

dead-code and direction); zero confirmed issues. On grinding: a proposer can only influence the hash among blocks that already tie on work, certification, and VRF, so it buys nothing beyond resolving that rare tie, with no safety impact.

Not yet rolled out to the committee. Since it is a consensus change I am holding the deploy for your go-ahead and a coordinated rolling restart, the same way we did 4a. It is node-local and non-forking (old and new nodes interoperate), so the rollout can be incremental with no fork risk.